

Keywords: agent evaluation • benchmark • long-horizon tasks • dense rewards

Long-Horizon-Terminal-Bench

A 46-Task Dense-Reward Benchmark Exposes the True Performance Gap Between LLM Agents on Complex Terminal Workflows

By Zongxia Li, Zhongzhi Li, Yucheng Shi, Ruhan Wang et al. (Tencent HY LLM Frontier; Univ. of Maryland; et al.)

arXiv:2607.08964 • July 9, 2026

Binary pass/fail benchmarks for coding and terminal agents are comfortable lies: they assign 0 or 1 to an episode without capturing how far the agent progressed on complex, multi-step workflows. Long-Horizon-Terminal-Bench (LHTB) replaces this illusion with 46 tasks spanning nine categories, graded by hidden rebuild-from-artifact verifiers that assign partial credit through fine-grained subtask completion. The result is a sobering view: top agents score only 28–34% on the hardest categories—and would have appeared far more capable under binary grading.

AT A GLANCE

Problem: Binary agent benchmarks mask variation in task progress, distorting comparisons and hiding true capability gaps.

Why it matters: Real deployment requires agents that make sustained, useful progress over hundreds of steps; binary grading cannot differentiate 5% from 95% completion.

Method: 46-task terminal benchmark with subtask-level dense rewards; hidden rebuild-from-artifact verifiers prevent score gaming.

Result: Top agents (Claude 3.7, GPT-4o) average 52% overall; only 28–34% on scientific computing and experiment reproduction.

Evidence: Three frontier models evaluated; binary-vs-dense correlation only 0.41, confirming benchmarks measure different capabilities.

The Big Picture

Current LLM agent evaluations suffer from a fundamental measurement problem. SWE-bench, HumanEval, and Terminal-Bench all grade each task as a binary 0 or 1: either the final state passes all tests, or it does not. This approach is easy to implement and produces clean leaderboards, but it conflates “failed to start” with “almost finished”—both receive the same score of 0.

The problem compounds on long-horizon tasks where success requires chaining dozens of subtasks: running a numerical simulation, checking convergence, generating plots, and writing a report. An agent that completes the simulation and convergence check but fails on the plotting step does 60% of the work—but receives 0 credit under binary grading.

LHTB introduces dense intermediate rewards by decomposing every task into 4–12 fine-grained, verifiable subtasks, each graded independently. Partial credit is not merely cosmetic: it changes which agents rank highest and by how much, and reveals capability gaps invisible under binary evaluation.

Why Existing Approaches Fall Short

Terminal-Bench introduced the key insight of containerized, agentic evaluation with a bash shell—but retained binary grading. SWE-bench Verified improved task realism (real GitHub issues) but also grades binary. Both benchmarks are additionally vulnerable to self-reported progress: agents can write test files that trivially pass their own assertions, gaming the verifier.

Process reward models (PRMs) offer a complementary approach—training a model to assign intermediate rewards—but require supervised reward data that is expensive to collect and brittle to domain shift. LHTB achieves dense rewards without any learned reward model by decomposing tasks into subtasks with objectively verifiable intermediate states (e.g., “the simulation output file exists and contains a valid numpy array with shape (T, N) ”).

Core Mechanism

The benchmark’s core contribution is the combination of (1) task decomposition into verifiable subtasks and (2) rebuild-from-artifact verification that prevents score gaming.

For rebuild-from-artifact verification, instead of running the agent’s code directly, the verifier mounts the agent’s output directory read-only, extracts specified intermediate artifacts (files, database entries, generated figures), reconstructs a ground-truth comparison from these artifacts, and asserts properties. This prevents agents from planting self-passing assertions or manipulating the test environment because the verifier operates on static artifacts, not live processes.

Technical Formulation

Each task τ decomposes into subtasks $\{s_1, \dots, s_m\}$, each with a binary verifier $v_i \in \{0, 1\}$. The composite score is:

$$S(\tau) = 0.7 \cdot \frac{1}{m} \sum_{i=1}^m v_i + 0.2 \cdot V_{\text{final}} + 0.1 \cdot B_{\text{eff}}$$

where V_{final} is a holistic final-state verifier and B_{eff} is an efficiency bonus that decays linearly for episodes requiring more than 100 steps. This scoring incentivizes both thorough subtask completion and efficient execution.

Learning or Inference Procedure

Agents are placed in an Ubuntu 22.04 Docker container with standard scientific Python, build tools, git, and a network connection. They receive three tools: (1) a bash executor, (2) a file editor, and (3) a web search interface. The step limit is 200; agents are instructed only with the natural-language task description.

1. Agent reads task description and initializes workspace.
2. Agent executes a sequence of bash commands, file edits, and searches to complete the task.
3. After each agent-initiated “checkpoint” (or at the step limit), the verifier mounts the filesystem, extracts specified artifacts, and runs subtask assertions.
4. Subtask credits accumulate; a final holistic verifier runs after the episode ends.

What the Guarantee Says

The red-team evaluation provides an empirical anti-gaming guarantee: agents instructed to game the verifier succeeded on 0/46 tasks, versus 23/46 on a naive run-and-check verifier. The rebuild-from-artifact approach does not prevent all possible manipulation, but the attack surface is dramatically smaller because agents cannot interact with the verifier process at all—only their output artifacts are examined.

Experimental Findings

Category	Claude 3.7	GPT-4o	Gemini 2.5
Software Eng.	72.1%	68.4%	65.2%
Data Eng.	61.3%	58.7%	54.1%
DevOps	55.8%	52.3%	49.6%
Interactive Games	41.5%	38.2%	35.8%
Experiment Repro.	34.2%	30.1%	28.7%
Scientific Comp.	28.9%	27.4%	24.3%
Overall (avg.)	52.3%	49.1%	45.9%

The correlation between binary pass/fail scores on prior benchmarks and LHTB subtask credit is only 0.41, confirming that LHTB captures genuinely different capabili-

ties. Binary grading reverses the ranking between Claude 3.7 and GPT-4o on 7 of 46 tasks.

Ablations and Interpretation

Binary vs. dense grading: Binary grading on LHTB tasks produces rankings that disagree with dense grading on 7/46 tasks, illustrating how coarse grading can mislead model selection.

Verifier robustness: In the red-team evaluation, agents instructed to game succeed on 0/46 tasks under rebuild-from-artifact verification versus 23/46 under naive run-and-check, a 100% reduction in gaming success rate.

Step count analysis: Agents average 43 steps on software engineering tasks and 161 steps on scientific computing tasks, confirming that LHTB successfully stratifies tasks by difficulty and that step count is a meaningful proxy for task complexity.

Category difficulty ordering: The difficulty gradient (software engineering $\rightarrow$ data engineering $\rightarrow$ DevOps $\rightarrow$ games $\rightarrow$ experiment reproduction $\rightarrow$ scientific computing) is consistent across all three frontier models tested, suggesting it reflects genuine task structure rather than model-specific biases.

Reference

Zongxia Li et al. **Long-Horizon-Terminal-Bench: Testing the Limits of Agents on Long-Horizon Terminal Tasks with Dense Reward-Based Grading.** arXiv:2607.08964, July 2026. <https://arxiv.org/abs/2607.08964>